penny | procurement.AI

Frontend Engineering Take-Home

Change Request Review UI - Angular

Assessment repository Provided separately through the Google Drive link in your invitation email.

Welcome. This exercise mirrors a normal frontend slice at Penny. You will complete an existing Angular reviewer experience for Change Requests against a provided mock API contract. If you reach the time limit, stop and document what remains - thoughtful scoping is part of the signal.

At a glance

TIME BUDGET	WORKING WINDOW	STACK	PRIMARY SIGNAL
10-16 focused hours	**Across 4 days**	**Angular 15 + Jest**	**Explicit UI state and rendered behavior**

Your assignment

Build a clear reviewer flow that helps an approver find a Change Request, understand the proposed changes and history, and approve or reject it - while respecting user permissions and handling loading, empty, slow, and failing states.

The repository's `CANDIDATE_BRIEF.md` is the detailed source of truth. This PDF is the accompanying summary.

What to complete

1 **Orient and document**
Read the README, models, fixtures, and templates. Describe the screens and component state, plus assumptions, testing strategy, AI usage, and tradeoffs in `IMPLEMENTATION_NOTES.md`.

2 **Fix the starting failures**
Correct the diff/preview classification defect and the detail-page permission defect that offers Approve to an unauthorized user.

3 **Complete the list experience**
Keep loading, loaded, empty, and error states correct and implement the status filter so both `visibleRows` and the rendered table narrow correctly.

4 **Complete the detail experience**
Render an accurate diff with totals and delta, order the approval timeline chronologically, and keep action availability dependent on both status and policy.

5 **Implement safe actions**
Call the provided API for Approve/Reject, prevent invalid or duplicate actions, require a rejection reason, and preserve a coherent view during slow or failed calls.

6 **Add rendered-behavior tests**
Cover list states and filtering, diff and timeline behavior, permission gating, action flows, failure paths, and reject validation using component/DOM tests.

How your work will be evaluated

- All original and added tests pass from a clean install.
- The list has a working status filter and clear loading, empty, and error states.
- The detail view shows an accurate diff, totals/delta, and chronological timeline.
- Actions respect both CR status and user permissions; read-only users cannot act.
- Approve/Reject handle slow and failed responses without leaving stale UI state.

> Working principles. Keep the existing architecture and code style. Prefer clarity and correctness over cleverness. Use the provided error patterns and avoid new heavyweight dependencies unless you explain the reason.

Submission checklist

Repository	Submit a git repository with full commit history. Do not squash to a single commit.
Engineering notes	Include `IMPLEMENTATION_NOTES.md` at the repository root (aim for 1-2 pages).
Walkthrough	Provide a 5-8 minute screen recording. Walk through the list and detail screens, including a rejection and an error state, and explain one non-trivial frontend decision.
Clean run	Ensure `npm ci && npm test` works from a clean clone. Run build, lint, and formatting checks before submission.

AI tools

Allowed and expected. Briefly disclose where and how you used AI in `IMPLEMENTATION_NOTES.md`. You should understand and be able to explain and modify every part of your submission.

Follow-up conversation

Expect a 45-60 minute discussion of your component model, a small live UI change, and a debugging scenario around action state or network behavior.

Before sending. Follow the delivery instructions from your Penny recruiting contact, preserve full commit history, and call out any special setup or unfinished work clearly.

Thank you for the time and care you put into the exercise. We hope you enjoy working through it.